



*Software Application Development Tools & Techniques*  
*(Software Engineering)*

**By Dinesh Anantwar**

[danantwar@gmail.com](mailto:danantwar@gmail.com)

# Introduction : Dinesh Anantwar

- 23+ years of Global IT industry experience
- Worked with,
  - Fujitsu ICIM (1 year) – Hardware Industry
  - CDAC R&D (1.5 years) – Systems & Parallel Programming
  - Infosys (10 years) –
    - Worked on Projects in USA, Germany, Malaysia and India
    - Headed ‘Education and Research’ for Infosys-Pune, 2003-5
  - iGATE Patni (6 years)
    - Globally Headed ‘Training Delivery and Certification’ at iGATE Patni 2006-12
  - Mentor/Consultant (5+ years) ##



# Introduction : Dinesh Anantwar

- From 2012, Mentoring/Consulting to,
  - IT companies like TCS, Amdocs, Fundtech, EValueServe, Tata Power, FIS Global, Credit Suisse, Vodaphone etc.
  - CDAC ACTS, MBA, M. tech and Engineering college Faculties n students
- Career Mentor for all 8000+ employees of Amdocs India a for 3 years in 2013-14-15. Mentored 600+ employees including Directors, Sr. Managers though one to one closed door confidential meetings.

# Introduction : Dinesh Anantwar

- Conducted training sessions at,
  - USA (New York, Seattle, Houston, Hartford, Philadelphia, Richmond)
  - UK (Manchester)
  - India (Bangalore, Chennai, Hyderabad, Mumbai, Delhi, Ahmedabad, Pune, Mangalore, Mysore)
- Conducts wide variety of Project Management and Technical Trainings
- Training Trainers to Teach effectively ##



# Introduction : Dinesh Anantwar

- Academics,
  - B. E. (Electronics) Pune University
  - Engineering Post graduation (Software) : BITS Pilani  
Post Graduate Diploma in Advance Computing ACTS-  
CDAC, Pune
  - Management Post Graduation : IIM-K
  - Certifications
    - PMP from PMI-USA
    - IBM certified Expert in OO Analysis and Design (RUP)
    - Certified SCRUM Master
- “Board of Studies” Member at Symbiosis International  
University from 2004 to 2012 ##

# Personal Introduction

- Attended Military training in Bhosala Military school
- I love Adventure – Trekking, Rock Climbing, Rapelling, Travelling, Kayaking, Rafting, Scuba diving...
- Visited 200+ forts in Maharashtra
- Did Himalayan Trek of Kanchanganga base Camp in 2011
- Rock climbed Lingana, Karthik, Padargad, Kalakrai and Telbaila pinnacles
- Did Solo paragliding from 1000 ft. in 2012. Student Pilot Licence.
- Pune to Bangalore by Car (875km) in 12.5 hrs 2013
- Did 2700km Bullet ride in Himalayas (Himachal, Ladakh, Kashmir, Punjab) 2014
- Bike Ride to Spiti Valley in July 2016
- Facebook : 8300+ friends
- LinkedIn 5000+ connections
- Regular Blood Donor
- Teach for free to Rural/Poor/Needy schools and colleges ##



# Teaching at Manchester, England





# At a Management Conference





## Teaching Principles, HoDs and Professors on “How to Teach”

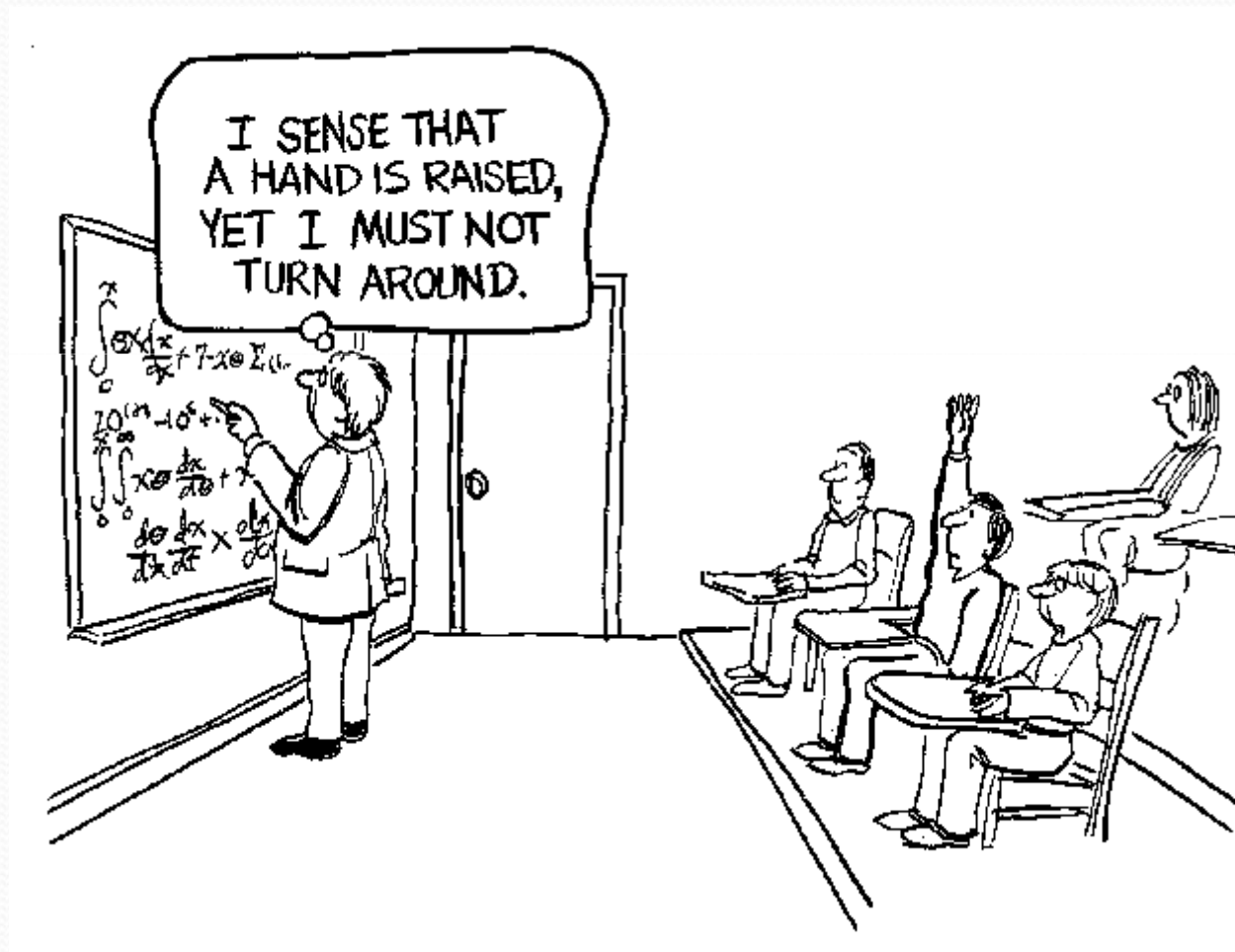


# Approach of Training

- A very direct and practical approach to develop required skills and impart useful knowledge
- A lot of,
  - Real life examples
  - Scenario/Case Study based Discussions – More you participate more you learn
  - Hands on practice
  - Best Practices
  - Typical Mistakes n ways of avoiding those
  - Role Plays ##



**Do ask questions....**



**There will be One designated break  
after Every 1.5 hour during my session**





## Course Contents

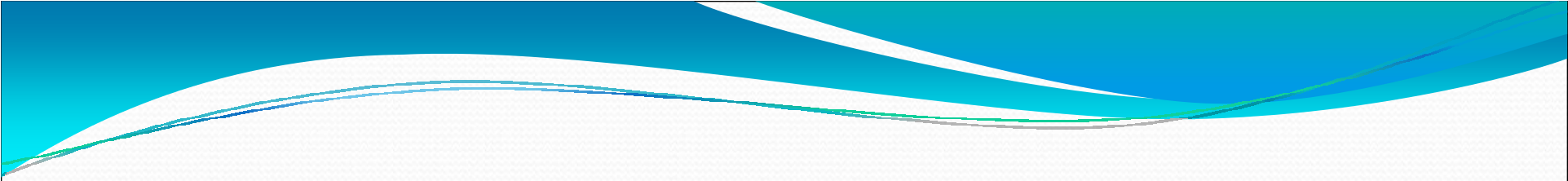
- Software Engineering, Project Management
- Software life cycle and various life cycle models
- Requirements Management
- Effort, Schedule and Cost Estimation
- Design – OO Mindset, RUP, UML, STARuml
- Development, Closure and Maintenance



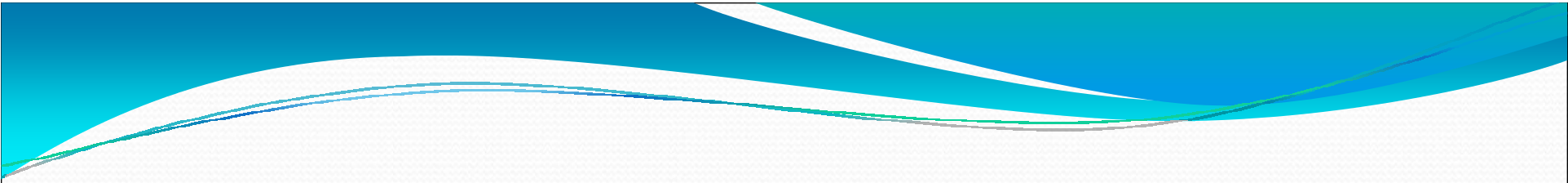
## Course Contents

- Testing
- Agile Methodology – SCRUM
- Software Quality Assurance
- Risk Management
- Trends in Software Engineering





# **Introduction To Software Engineering**

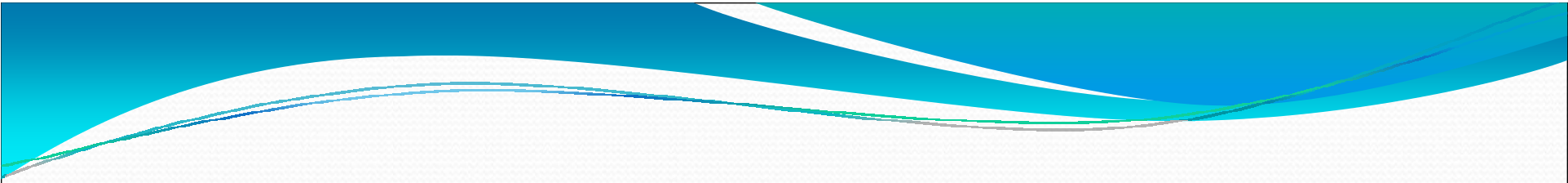


# Bungalow and Building Example

Software programs have exponential growth in complexity and difficulty level with respect to size.

The **ad hoc** (unstructured) approach which works on developing small programs breaks down i.e. fails when size of software increases.





## Symptom of Software Crisis

- About **US\$250 billions** spent per year in the US on application development
- Out of this, about **US\$140 billions** wasted due to the projects getting abandoned or reworked; this in turn because of not following best practices and standards

**Ref: Standish Group**

# Symptom of Software Crisis.. More statistics

- 10% of client/server apps are abandoned or restarted from scratch
- 20% of apps are significantly altered to avoid disaster
- 40% of apps are delivered significantly late

**Source: 3 year study of 70 large c/s apps 30 European firms.  
Compuware**



# Observed Problems

- Software products:
  - fail to meet user requirements
  - crash frequently
  - expensive
  - difficult to alter, debug, enhance
  - often delivered late
  - use resources non-optimally

# Confused with Programs and Products

## Programs

- Usually small in size
- Author himself is sole user
- Single developer
- Lacks proper user interface
- Lacks proper documentation
- Ad hoc development.

## Software Products

- Large
- Large number of users
- Team of developers
- Well-designed interface
- Well documented & user-manual prepared
- Systematic development



# Challenges in Software Development

- False assumptions : 3 second for web page example
- Rapidly changing requirements : e.g PrePaid
- Pressures for faster time to market : e.g Mobile
- Continuing rapid evolution of software methodologies and systems. Not everyone is able to Adopt/Cope up with it.

# Geographical nature of Software Projects

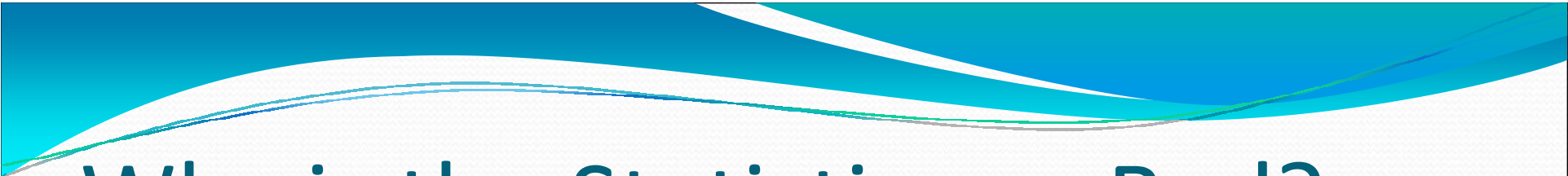
- Software created for Bank of America. In which country will it be used?
  - America only? No! World wide. In Pune too
  - Software need to work across various countries, languages and cultures. E.g Facebook, Twitter ##



# Three key Challenges

Software engineering in the 21<sup>st</sup> century faces three key challenges:

- **Legacy systems**
  - Old, valuable systems must be maintained and updated
- **Heterogeneity**
  - Systems are distributed and include a mix of hardware and software
- **Delivery**
  - There is increasing pressure for faster delivery of software



# Why is the Statistics so Bad?



# Software Programming ≠ Software Engineering

- **Software programming i.e. What we did in College:** the process of translating a problem from its physical environment into a language that a computer can understand and obey. (*Webster's New World Dictionary of Computer Terms*)
  - Single developer
  - "Toy" applications
  - Short lifespan
  - Single or few stakeholders
    - Architect = Developer = Manager = Tester = Customer = User
  - One-of-a-kind systems
  - Built from scratch
  - Minimal maintenance

# Software Programming ≠ Software Engineering

- Software engineering i.e. What we need to do in Industry
  - Teams of developers with multiple roles
  - Complex systems
  - Indefinite lifespan
  - Numerous stakeholders
    - Architect ≠ Developer ≠ Manager ≠ Tester ≠ Customer ≠ User
  - Maintenance accounts for over 60% of overall costs



# Why is the Statistics so Bad?

- Software professionals lack training on scientific approach to sw development
  - Programmers have skills for programming but without the engineering mindset about a process discipline they fail

# Software Myths

## (Customer Perspectives)

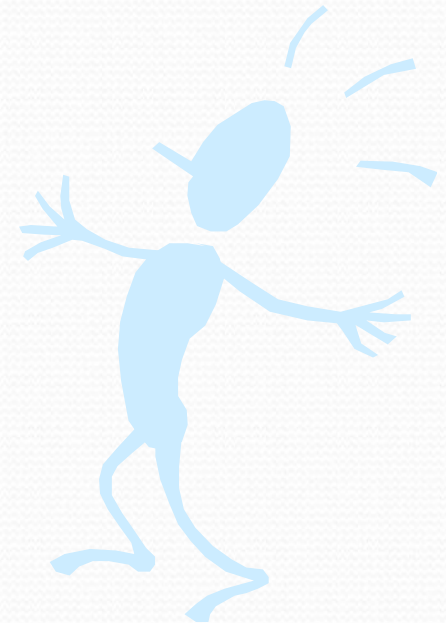
- A general statement of objectives is sufficient to get started with the development of software. Missing/vague requirements can easily be incorporated/detailed out as they get concretized.
- Application requirements can never be stable; software can be and has to be made flexible enough to allow changes to be incorporated as they happen. ##



# Software Myths (Developer Perspectives)

Once the software is (Designed, Developed, Tested and then) deployed, the job is done.

*Usually, the problems just begin!*

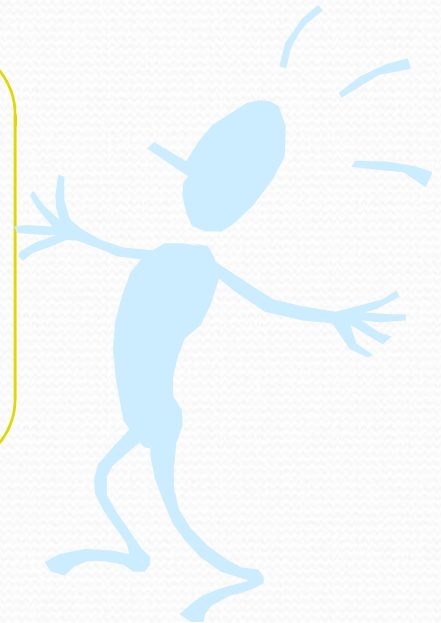


# Software Myths

## (Developer Perspectives)

Until the software is coded and is available for testing, there is no way for assessing its quality.

*Usually, there are too many tiny bugs inserted at every stage that grow in size and complexity as they progress thru further stages!*





# Software Myths (Developer Perspectives)

The only deliverable for a software development project is the tested code.

*The code is only  
the externally visible component  
of the entire software complement!*



# Software Myths (Management Perspectives)

As long as there are good standards and clear procedures in my company, I shouldn't be too concerned.

*But the proof of the pudding  
is in the eating;  
not in the Recipe !*





# Software Myths (Management Perspectives)

As long as my software engineers have access to the fastest and the most sophisticated computer environments and state-of-the-art software tools, I shouldn't be too concerned.

*The environment is  
only one of the several factors  
that determine the quality  
of the end software product!*



# Software Myths

## (Management Perspectives)

When my schedule slips, what I have to do is to start a fire-fighting operation: add more software specialists, those with higher skills and longer experience - they will bring the schedule back on the rails!

*Unfortunately,  
software business does not  
entertain schedule compaction  
beyond a limit!*





# Unique Characteristics of Software

- Software is **malleable (flexible)**
- Software construction is **human-intensive**
- Software is **intangible** and **hard to measure**
- Software problems are usually **complex**
- Software directly depends upon the **hardware**
  - It is at the top of the system engineering “food chain”
- Software **doesn't wear out** but will deteriorate
- So Software solutions require **unusual thoroughness**

# What is Software Engineering?

- Different focuses for this term exist in various textbooks. Some are listed below.
- The application of a systematic, disciplined, quantifiable approach to development, operation, and maintenance of software; that is, **the application of engineering to software**. (*IEEE Standard Computer Dictionary*, 610.12, ISBN 1-55937-079-3, 1990)



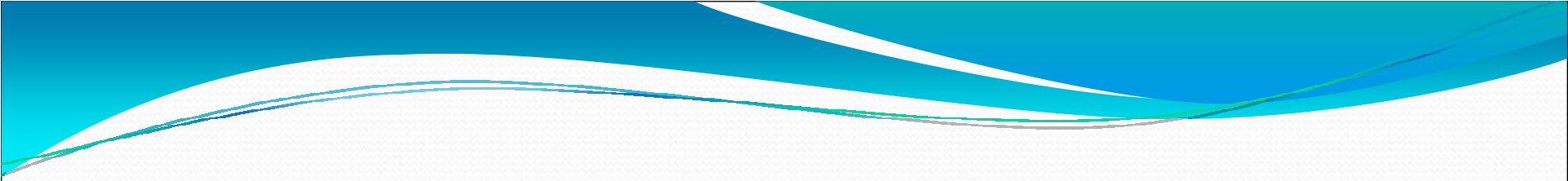
# What is Software Engineering? (contd)

- Multi-person construction of multi-version software (Parnas, 1987)
- A discipline that deals with the building of software systems which are so large that they are built by a team or teams of engineers (Ghezzi, Jazayeri, Mandrioli)

# Software Engineering

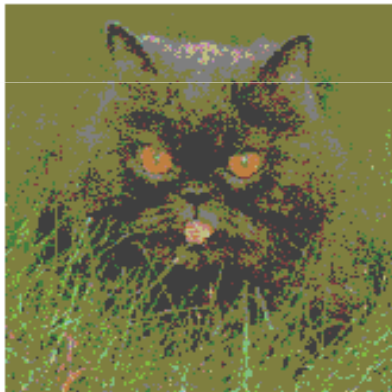
- It is nothing but a set of best practices
- Where so best practices come from?
  - From experience of excellent people
  - Helps fresher or less experienced people perform much better like excellent experienced people ##





# Just for Laughs

# What is Software Engineering?



- Mechanical Engineering is like looking for a black cat in a lighted room.
- Chemical Engineering is like looking for a black cat in a dark room.
- Software Engineering is like looking for a black cat in a dark room in which there is no cat.
- Systems Engineering is like looking for a black cat in a dark room in which there is no cat and someone yells, "I got it!"



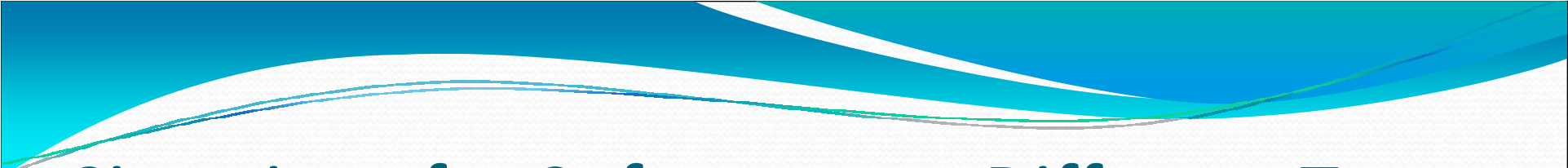
# Not Crisis, but a Chronic Problem

- The crisis persists
  - After 35 years later, the software “crisis” is still with us
  - Major problems are still the same:
    - poor quality (correctness, usability, maintainability, etc)
    - over budget
    - delivered late, or not at all
- It is not a crisis but a **chronic problem**
  - It becomes a persistent, chronic condition that software industry has to face with

# Situation of Software Engineering Today?

- Manager-Senior Manager Scenario, Woodcutter Scenario
- Everyone is **too busy** getting product out of the door to spend time in education or training or addressing these problems effectively
- “**Out of date**” **practices** become institutionalized ##





## Situations for Software are Different Too

- **Talent shortage:** needed software engineering skills often in short supply

## Why is Software Different?

- Software is **Engineered** or **Developed**, not **Manufactured** in the traditional sense
- Software doesn't "wear out" in the same sense as hardware
- Most Softwares are custom-build



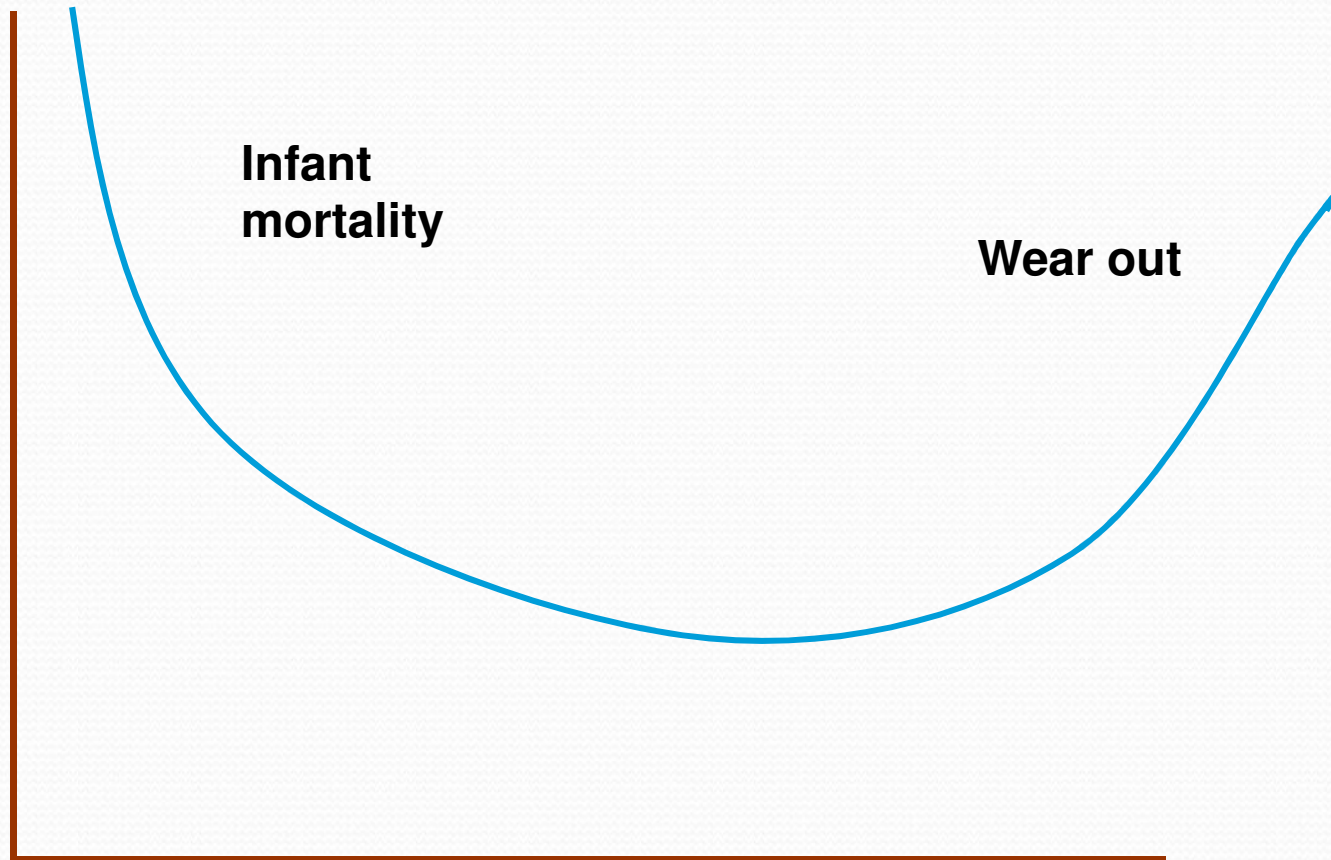
## Failure curve for hardware

Hardware  
Failure rate.

Infant  
mortality

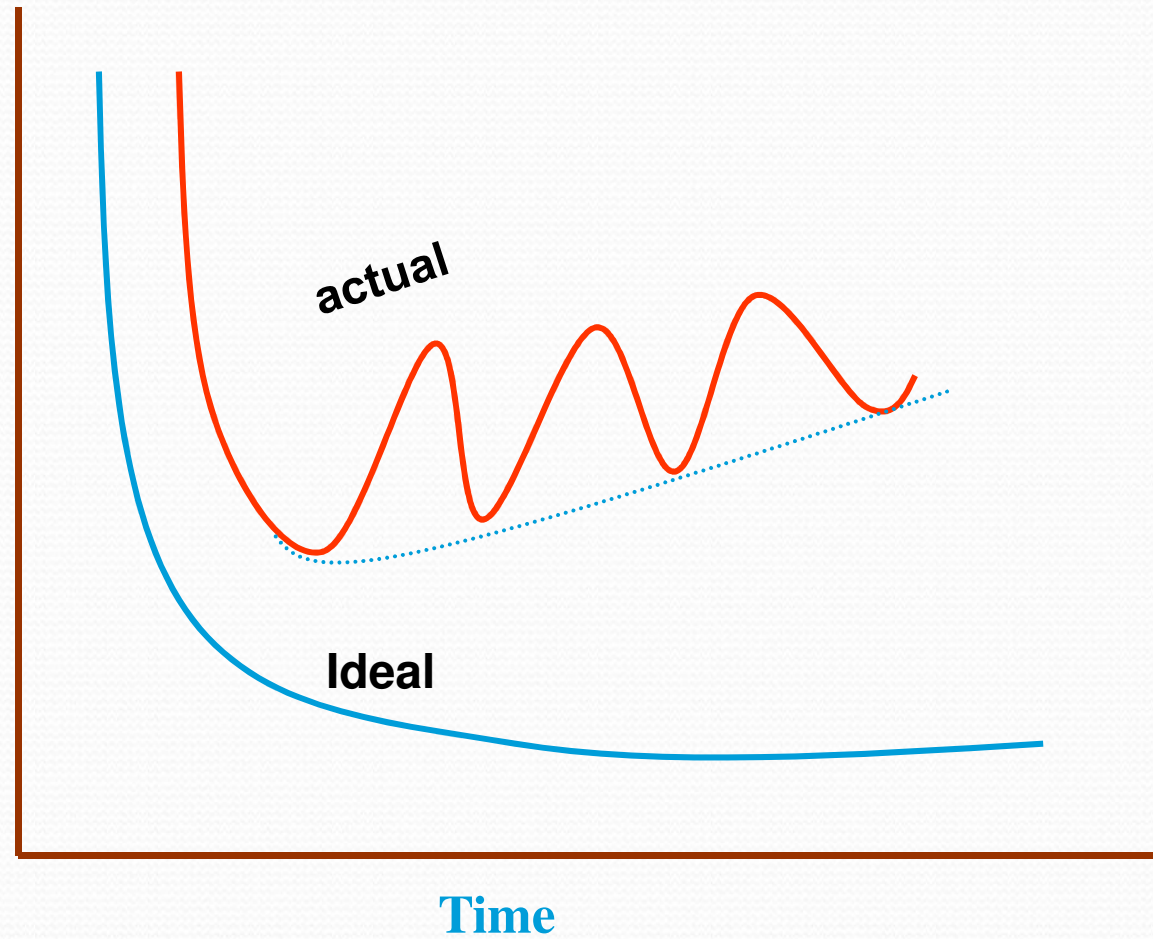
Wear out

Time

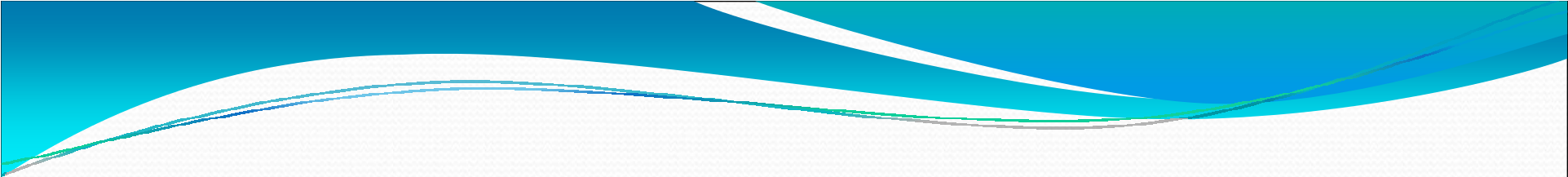


## Failure curve for software

Software  
Failure rate.



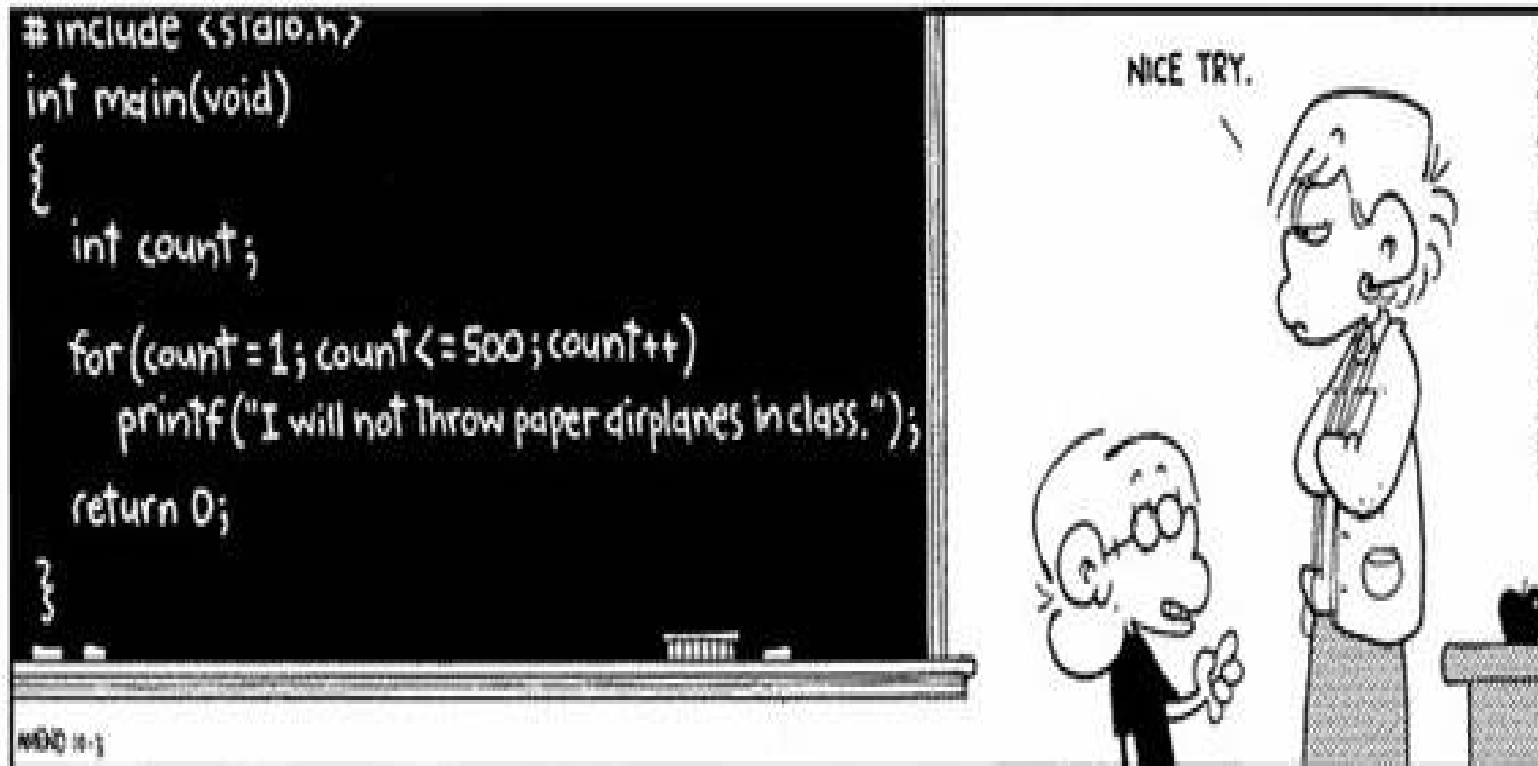




**Just for laughs**

## Be a born Software Engineer

**The Teacher gave a punishment to the student and asked him to write "I Will Not Throw Paper Airplanes in the Class" 500 times. AND the Student Wrote:**

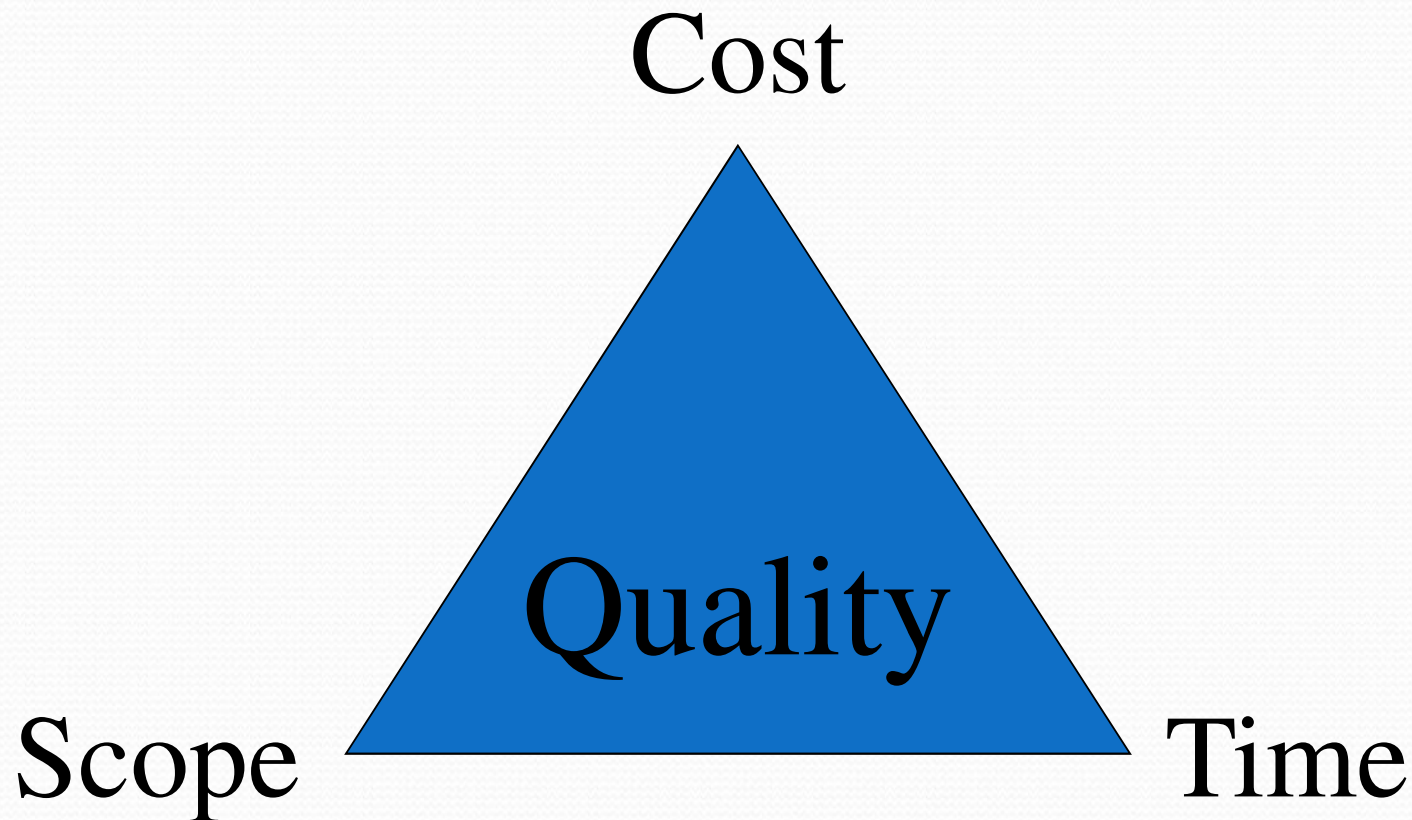




# What is a project? What is defn. of proj. success?

- A project is a TEMPORARY ENDEAVOR which PROGRESSIVELY ELABORATES a CONCEPT/IDEA and creates a UNIQUE Product or Service, using limited RESOURCES
- The Project is a SUCCESS if it meets the pre-defined OBJECTIVES within the pre-defined CONSTRAINTS

# The Project Management Triangle

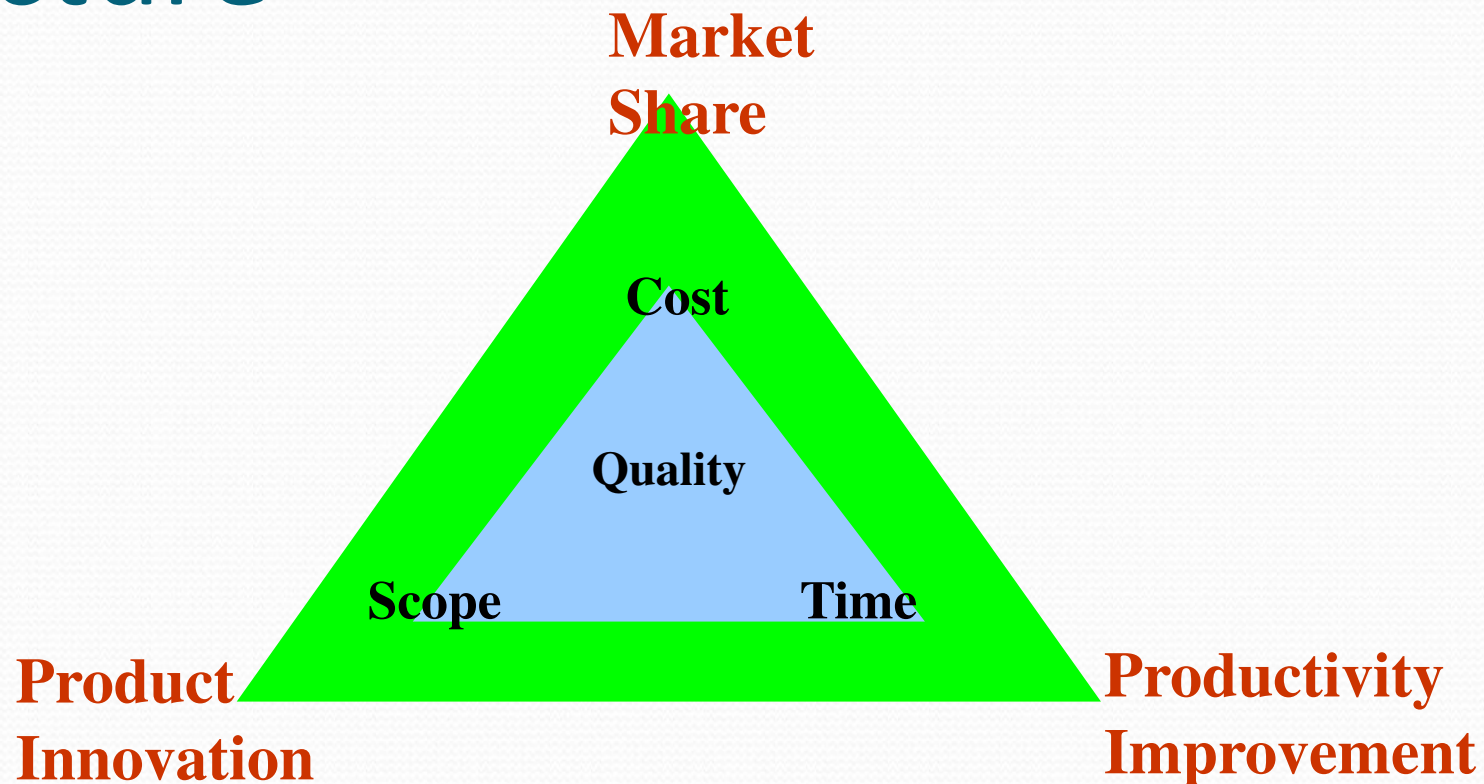




## How do you define success of a project?

- A large municipal corporation completed a SAP ERP implementation project for handling birth and death notifications. All functionalities are implemented, in time, within given cost with required quality. A successful project, isn't it?
- Earlier it used to take 1-2 hours to issue birth and death certificates; Now it takes 2 days!! Would you call this project a success? Why or why not? What could be the reasons? ##

# Project Management – The Big Picture



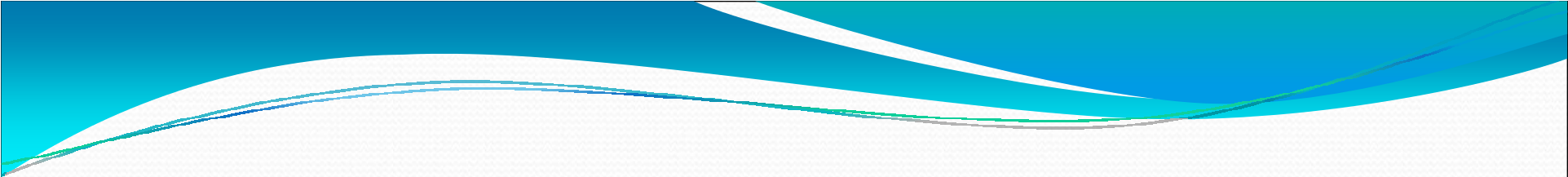
**Project Management – Need of the hour**

**Services to Solutions**



# Goals of Software Engineering ...

- production of **quality** software,
- delivered **on time**,
- **within budget**,
- satisfying **customers' requirements** and **users' needs**



**End of session**